DateTime is a subclass of Date and is generally considered deprecated in favor of plain old Time unless you specifically need its calendar calculation facilities. In this section we'll talk about Date mostly, since DateTime is a subclass, so you can assume anything about Date also applies to DateTime.

The main method for creating a Date is with Date.today, which returns the current date as a Date instance. You'll also often see Date.new(year, month day), or Date.parse("2023-2-27")—the parse method uses similar logic to Time.parse. There's also a Date.strptime that takes a format string, again, just like Time.strptime. The DateTime class has DateTime.now and also a DateTime.new that takes in arguments similar to the multi-argument format of Time.new

Date can be the left side of an addition operation. The second operand is a number in days to be added to the Date.

```
require "date"
d = Date.new(2023, 2, 12)
d         # => #<Date: 2023-02-12 ((2459988j,0s,0n),+0s,2299161j)>
d + 25    # => #<Date: 2023-03-09 ((2460013j,0s,0n),+0s,2299161j)>
```

Date can be the left side of a subtraction operation. Subtraction either takes a numeric and returns a new Date that many days earlier, or another Date and returns the number of days in between the two of them.

Date also takes <<, which adds a number of months to the date, as in Date.today << 3.

Date also supports <=> and Comparable.

You can get a following day with Date#next, Date#next_day, Date#next_month, Date#next_year. The last three take an optional argument for how may steps you want to move forward. You also have prev_day, prev_month, and prev_year.

Once you have a Date, the components are accessible as year, month (aliased as mon), and day (aliased as mday). All the weekday predicate methods like sunday? also exist, as do wday (day of week), and yday (day of year). The DateTime class also adds hour, minute (aliased as min), second (aliased as sec), second_fraction (aliased as sec_fraction), and zone.

Date has most of the same string methods as Time, including ctime and strftime, and also implements deconstruct_keys for pattern matching. There's also to_datetime and to_time.

Math

The Numeric class and its subclasses handle a lot of the basic arithmetic in Ruby. (For more see Numbers, on page 502.) But sometimes you need to do more advanced math. The Math module provides a couple dozen math functions, largely trigonometry, but it also handles other branches of advanced math. The BigMath model recreates a subset of those methods for BigDecimal arguments and values.

All of these methods are module methods, so they are all called with module syntax, as in: Math.cos(value).

The Math module contains multitudes, especially if you like trigonometry. Specifically, it contains:

- The constant values Math::Pi and Math::E.
- sqrt(x) and cbrt(x), which return the square and cube roots of the argument, respectively.